 [How to use Learning Curves to Diagnose Machine Learning Model Performance](#)

[Click to Take the FREE Deep Learning Performance Crash-Course](#)

Search...

Stacking Ensemble for Deep Learning

Neural Networks in Python

Q

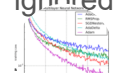
# Impact of Dataset Size on Deep Learning Model Skill And Performance Estimates

 [How To Improve Deep Learning Performance](#)  
Brownlee on January 2, 2019 in [Deep Learning Performance](#)

[Tweet](#) [Share](#) [Share](#)

 [How to use Data Scaling Improve Deep Learning Model Stability and Performance](#)  
Updated on August 25, 2020

Supervised learning is challenging, although the depths of this challenge are often learned then forgotten or willfully ignored.

 [Gentle Introduction to the Adam Optimization Algorithm for Deep Learning](#)

Just be the case, because dwelling too long on this challenge may result in a pessimistic outlook. In spite of the challenge, we continue to wield supervised learning algorithms and they perform well in practice.

Fundamental to the challenge of supervised learning, are the concerns:

## Loving the Tutorials?

- How can we reasonably approximate the unknown underlying mapping function from inputs to outputs? where you'll find the **Really Good** stuff.
  - How much data is needed to reasonably estimate the performance of an approximate of the mapping function?
- >> SEE WHAT'S INSIDE

Generally, it is common knowledge that too little training data results in a poor approximation. An over-constrained model will underfit the small training dataset, whereas an under-constrained model, in turn, will likely overfit the training data, both resulting in poor performance. Too little test data will result in an optimistic and high variance estimation of model performance.

It is critical to make this “*common knowledge*” concrete with worked examples.

In this post, we will work through a detailed case study for developing a Multilayer Perceptron neural network on a simple two-class classification problem. You will discover that, in practice, we don't have enough data to learn the mapping function or to evaluate models, yet supervised learning algorithms like neural networks remain remarkably effective.

- How to analyze the two circles classification problem and network learning algorithm.
- How changes in the size of a training dataset directly impact by neural networks.
- How the changes in the size of a test dataset directly impact a fit neural network model.

**Kick-start your project** with my new book [Better Deep Learning](#) [source code](#) files for all examples.

## Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

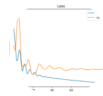
Email Address

START MY EMAIL COURSE

Let's get started  
Never miss a tutorial:



### Picked for you:



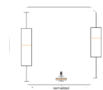
How to use Learning Curves to Diagnose Machine Learning Model Performance



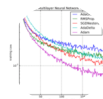
Stacking Ensemble for Deep Learning Neural Networks in Python



How To Improve Deep Learning Performance



How to use Data Scaling Improve Deep Learning Model Stability and Performance



Gentle Introduction to the Adam Optimization Algorithm for Deep Learning  
Impact of Dataset Size on Deep Learning Model Skill And Performance Estimates

Photo by Eneas De Troya, some rights reserved.

## Tutorial Overview

### Loving the Tutorials?

This tutorial is divided into five parts; they are:

The [Better Deep Learning](#) eBook is

where you'll find the **Really Good** stuff.

1. Challenge of Supervised Learning
2. Intro to the Circles Problem
3. Neural Networks
4. Study Test Accuracy vs Training Set Size
5. Study Test Set Size vs Test Set Accuracy

>> SEE WHAT'S INSIDE

## Introduction to the Circles Problem

As the basis for our exploration, we will use a very simple two-class or binary classification problem.

The scikit-learn library provides the [make\\_circles\(\)](#) function that can be used to create a binary classification problem with the prescribed number of samples and statistical noise.

Each example has two input variables that define the x and y coordinates. The points are arranged in two concentric circles (they have the same radius).

The number of points in the dataset is specified by a parameter. Gaussian noise can be added when sampling the points via the standard deviation of the noise, where 0.0 indicates no noise or points clustered tightly. A [pseudorandom number generator](#) can be specified via the "random\_state" parameter to be sampled each time the function is called.

### Start Machine Learning

### Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

The example below generates 100 examples from the two circles with no noise and a value of 1 to seed the pseudorandom number generator.

```
1 # example of generating circles dataset
2 from sklearn.datasets import make_circles
3 # generate circles
4 X, y = make_circles(n_samples=100, noise=0.0, random_state=1)
5 # show size of the dataset
6 print(X.shape, y.shape)
7 # show first few examples
8 for i in range(5):
9     print(X[i], y[i])
```

Running the example generates the points and prints the shape of the input (X) and output (y) components of the dataset. We can see that there are 100 examples of inputs with two features per example for the x and y coordinates and a matching 100 examples of the output variable or class value with 1 variable.

The first five examples from the dataset are shown. We can see that the x and y components of the input variables are generated on 0.0 and have the bounds [-1, 1]. We can also see that the class values are integers for either 0 or 1, indicating that examples are shuffled between the classes.

```
1 (100, 2), (100,)
2 [-0.6472136 -0.4702282] 1
3 [-0.34062349 -0.72386164] 1
4 [-0.53582679 -0.84432793] 0
5 [-0.5831749 -0.54763768] 1
6 [ 0.50993919 -0.61641059] 1
```

## Dataset

We can re-run the example and always get the same “*randomly generated*” points given the same pseudorandom number generator seed.

### Loving the Tutorials?

The example below generates the same points and plots the input variables of the samples using a scatter plot. We can use the `scatter()` matplotlib function to create the plot and pass in all rows of the X array with the first variable for the x coordinates and the second variable for the y coordinates on the plot.

>> SEE WHAT'S INSIDE

```
1 # example of creating a scatter plot of the circles dataset
2 from sklearn.datasets import make_circles
3 from matplotlib import pyplot
4 # generate circles
5 X, y = make_circles(n_samples=100, noise=0.0, random_state=1)
6 # scatter plot of generated dataset
7 pyplot.scatter(X[:, 0], X[:, 1])
8 pyplot.show()
```

Start Machine Learning

Running the example creates a scatter plot clearly showing the concentric circles of the dataset.

## Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

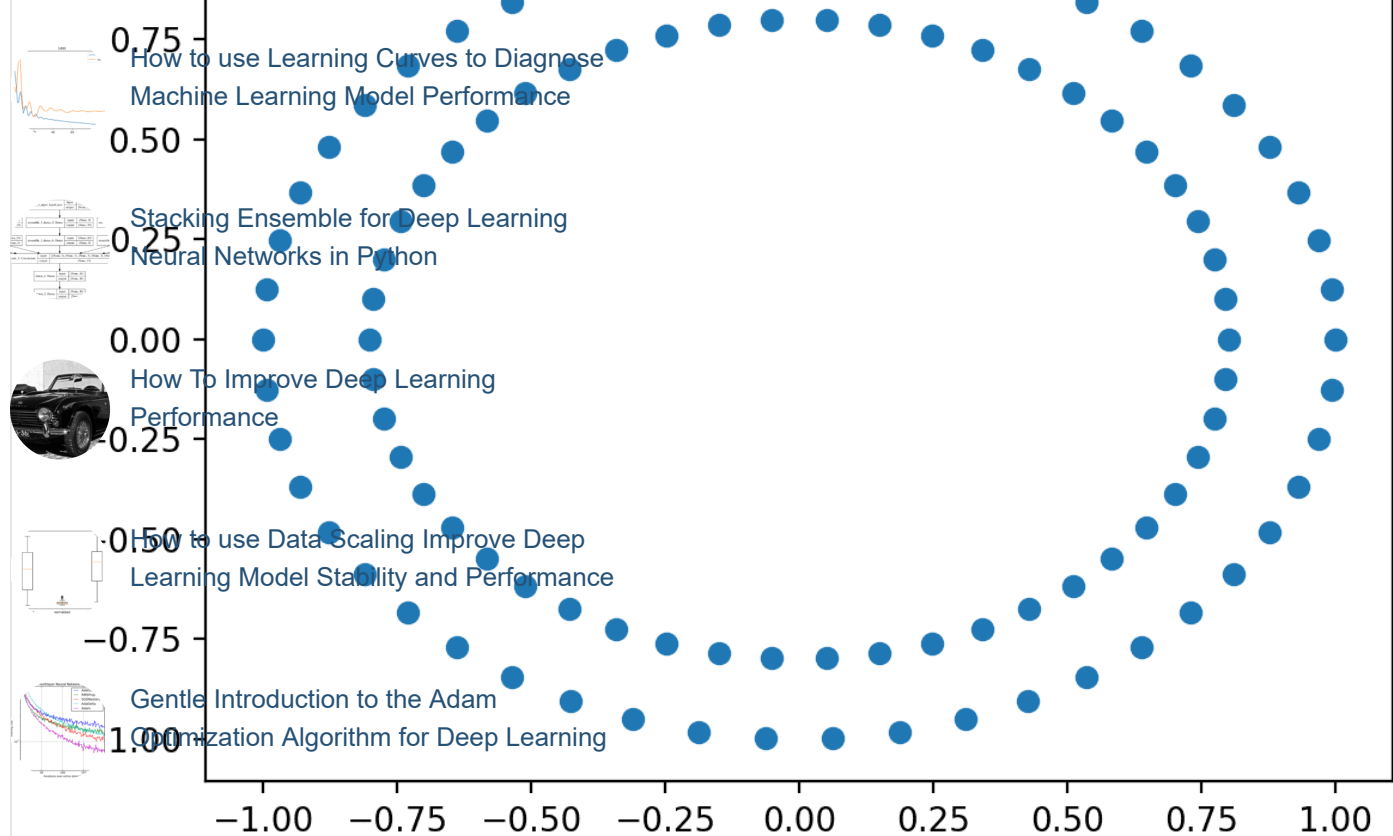
Email Address

START MY EMAIL COURSE

Never miss a tutorial:



Picked for you:



Loving the Tutorials?

The [Better Deep Learning](#) eBook is where you'll find the **Really Good** stuff.

We can re-create the scatter plot, but instead plot all input samples for class 0 blue and all points for class 1 red.

We can select the indices in the y array that have a given value using the `where()` NumPy function and then use those indices to select rows in the X array. The complete example is below.

```
1 # scatter plot of the circles dataset with points colored by class
2 from sklearn.datasets import make_circles
3 from numpy import where
4 from matplotlib import pyplot
5 # generate circles
6 X, y = make_circles(n_samples=100, noise=0.0, random_state=1)
7 # select indices of points with each class label
8 zero_ix, one_ix = where(y == 0), where(y == 1)
9 # points for class zero
10 pyplot.scatter(X[zero_ix, 0], X[zero_ix, 1], color='red')
11 # points for class one
12 pyplot.scatter(X[one_ix, 0], X[one_ix, 1], color='blue')
13 pyplot.show()
```

Running the example, we can see that the samples for class 0 (blue) belong to the inner circle and the samples for class 1 (red) belong to the outer circle.

Start Machine Learning

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

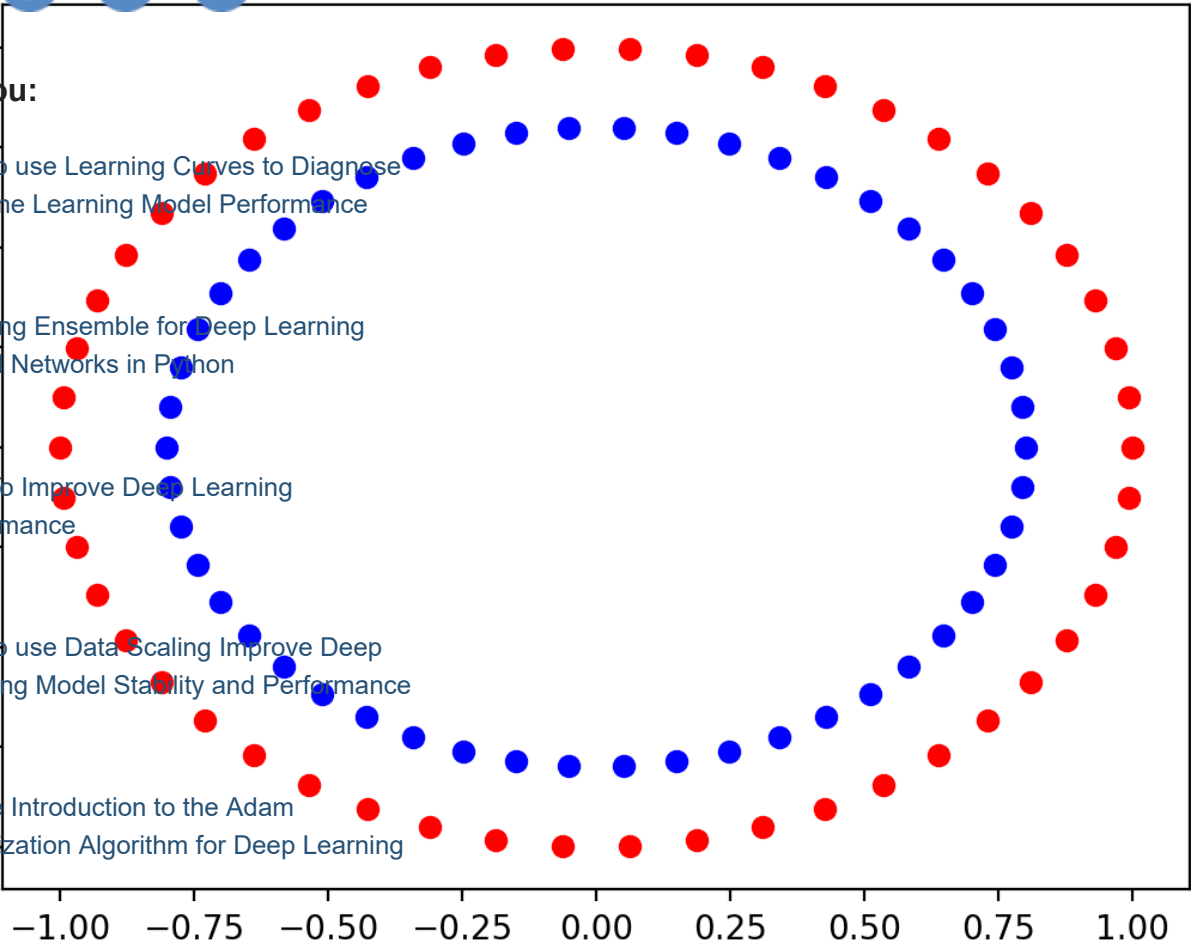
Email Address

START MY EMAIL COURSE

Never miss a tutorial:



Picked for you:



Loving the Tutorials?

The [Better Deep Learning](#) eBook is where you'll find the *Really Good* stuff.

Plot With Varied Noise

All real datasets have some statistical noise. More statistical noise means that the problem is more challenging for the learning algorithm to map the input variables to the output or target variables.

The circles dataset allows us to simulate the addition of noise to the samples via the “noise” argument.

We can create a new function called `scatter_plot_circles_problem` and creates a scatter plot with points colored by their class value.

```
1 # create a scatter plot of the circles dataset with the given amount of noise
2 def scatter_plot_circles_problem(noise_value):
3     # generate circles
4     X, y = make_circles(n_samples=100, noise=noise_value, random_state=1)
5     # select indices of points with each class label
6     zero_ix, one_ix = where(y == 0), where(y == 1)
7     # points for class zero
8     pyplot.scatter(X[zero_ix, 0], X[zero_ix, 1], color='red')
9     # points for class one
10    pyplot.scatter(X[one_ix, 0], X[one_ix, 1], color='blue')
```

We can call this function multiple times with differing amounts of noise to create a new problem.

We will create four scatter plots as subplots via the `subplot()` method with noise values [0.0, 0.1, 0.2, 0.3]. The complete example is listed below.

Start Machine Learning

Start Machine Learning

You can master applied Machine Learning without math or fancy degrees. Find out how in this free and practical course.

Email Address

START MY EMAIL COURSE



```

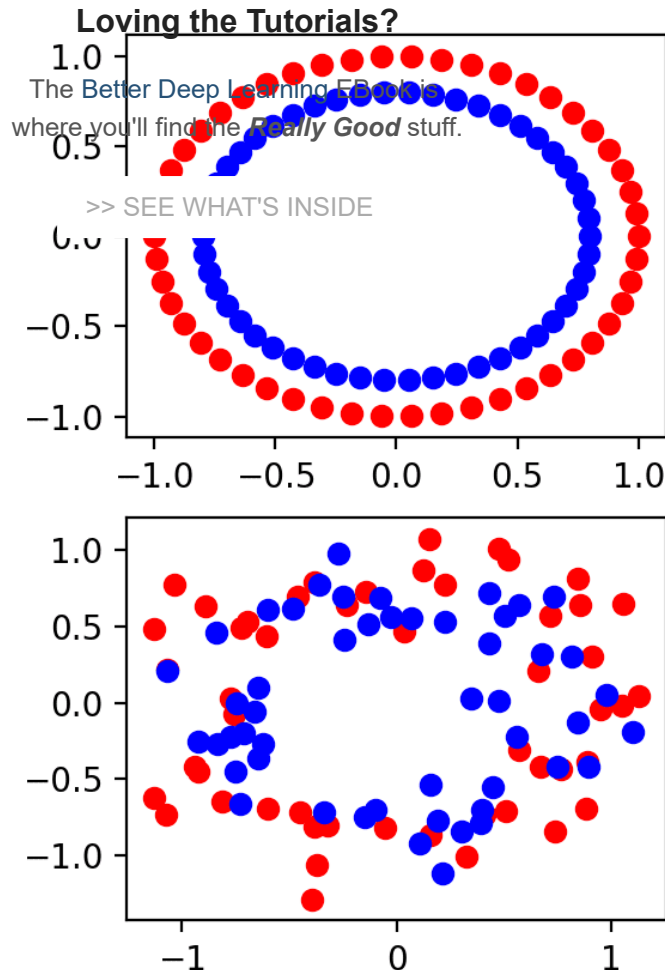
1 # scatter plots of the circles dataset with varied amounts of noise
2 from sklearn.datasets import make_circles
3 from numpy import where
4 from matplotlib import pyplot
5
6 # create a scatter plot of the circles dataset with the given amount of noise
7 def scatter_plot_circles_problem(noise_value):
8     # generate circles
9     X, y = make_circles(n_samples=100, noise=noise_value, random_state=1)
10    # select indices of points with each class label
11    zero_ix = where(y == 0)[0]
12    one_ix = where(y == 1)[0]
13    # points for class zero
14    pyplot.scatter(X[zero_ix, 0], X[zero_ix, 1], color='red')
15    # points for class one
16    pyplot.scatter(X[one_ix, 0], X[one_ix, 1], color='blue')
17
18 # vary noise and plot
19 values = [0.0, 0.1, 0.2, 0.3]
20 for i in range(len(values)):
21     value = 220 + (i+1)
22     pyplot.subplot(value)
23     scatter_plot_circles_problem(values[i])
24     pyplot.show()

```

Running the example, a plot is created with four subplots, one for each of the four different noise values [0.0, 0.1, 0.2, 0.3] in the top left, top right, bottom left, and bottom right respectively.

We can see that a small amount of noise 0.1 makes the problem challenging, but still distinguishable. A noise value of 0.0 is not realistic and a dataset so perfect would not require machine learning. A noise value of 0.2 makes the problem very challenging and a noise value of 0.3 may make the problem too challenging to learn.

Optimization Algorithm for Deep Learning



Four Scatter Plots of the Circles Dataset Varied

Start Machine Learning

Start Machine Learning

You can master applied Machine Learning without math or fancy degrees. Find out how in this free and practical course.

Email Address

START MY EMAIL COURSE

# Plot With Varied Sample Size

We can create a similar plot of the problem with a varied number of samples.

More samples give a learning algorithm more opportunity to understand the underlying mapping of inputs to outputs, and, in turn, a better performing model.

Picked for you:

We can update the `scatter_plot_circles_problem()` function to take the number of samples to generate as an input, as well as the amount of noise, and set a default for the noise of 0.1, which makes the problem noisy, but not too noisy.

```
1 # create a scatter plot of the circles problem
2 def scatter_plot_circles_problem(n_samples, noise_value=0.1):
3     # generate circles
4     X, y = make_circles(n_samples=n_samples, noise=noise_value, random_state=1)
5     # select indices of points with each class label
6     zero_ix, one_ix = where(y == 0), where(y == 1)
7     # points for class zero
8     pyplot.scatter(X[zero_ix, 0], X[zero_ix, 1], color='red')
9     # points for class one
10    pyplot.scatter(X[one_ix, 0], X[one_ix, 1], color='blue')
```

How to use Data Scaling Improve Deep Learning Model Stability and Performance

can this function to create multiple scatter plots with different numbers of points, spread evenly between the classes or classes.

We will try versions of the problem with the following sized samples [50, 100, 500, 1000]. The complete example is below.

```
1 # scatter plots of the circles dataset with varied sample sizes
2 from sklearn.datasets import make_circles
3 from numpy import where
4 from matplotlib import pyplot
5
6 # create a scatter plot of the circles problem
7 def scatter_plot_circles_problem(n_samples, noise_value=0.1):
8     # generate circles
9     X, y = make_circles(n_samples=n_samples, noise=noise_value, random_state=1)
10    # select indices of points with each class label
11    zero_ix, one_ix = where(y == 0), where(y == 1)
12    # points for class zero
13    pyplot.scatter(X[zero_ix, 0], X[zero_ix, 1], color='red')
14    # points for class one
15    pyplot.scatter(X[one_ix, 0], X[one_ix, 1], color='blue')
16
17 # vary sample size and plot
18 values = [50, 100, 500, 1000]
19 for i in range(len(values)):
20     value = 220 + (i+1)
21     pyplot.subplot(value)
22     scatter_plot_circles_problem(values[i])
23 pyplot.show()
```

Running the example creates a plot with four subplots, one for each sample size [50, 100, 500, 1000] in the top left, top right, bottom left, and bottom right respectively.

We can see that 50 examples are probably too few and even 100 is not enough. The plots suggest that 500 and 1,000 examples may be sufficient. However, that many “outlier” points result in overlaps between the two classes.

Start Machine Learning

## Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.**

Find out how in this *free* and *practical* course.

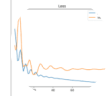
Email Address

START MY EMAIL COURSE

Never miss a tutorial:



Picked for you:



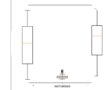
How to use Learning Curves to Diagnose Machine Learning Model Performance



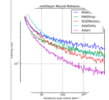
Stacking Ensemble for Deep Learning Neural Networks in Python



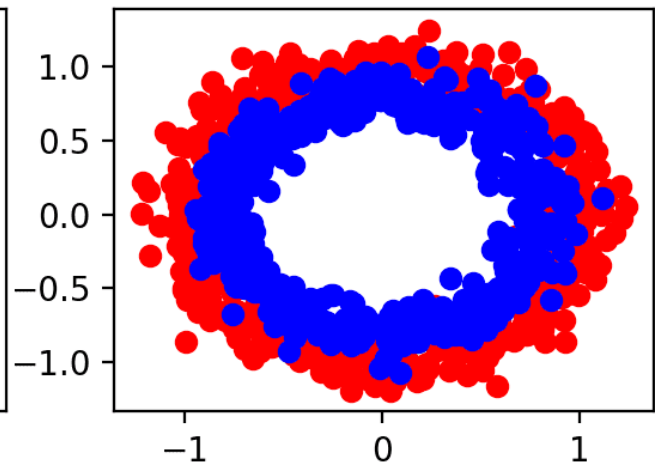
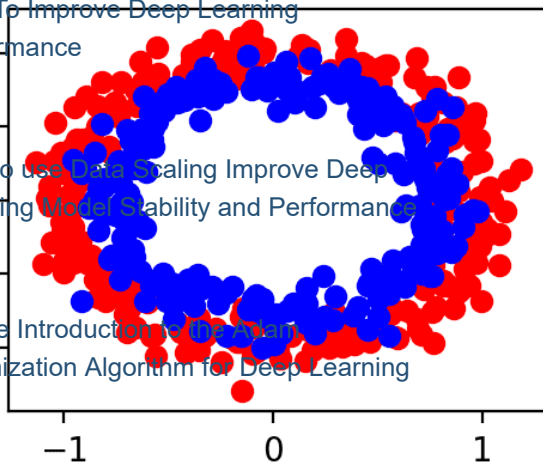
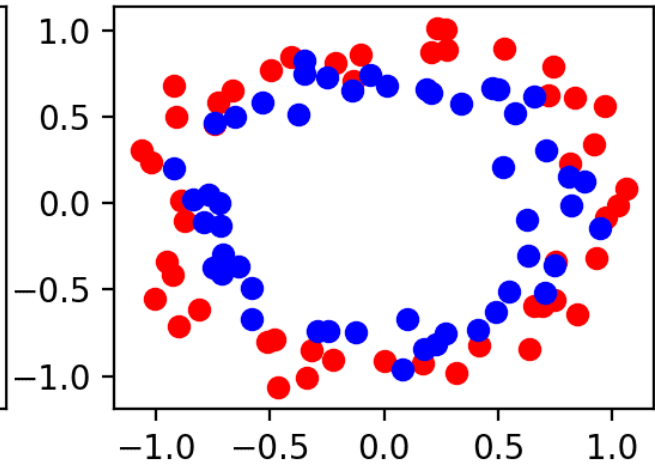
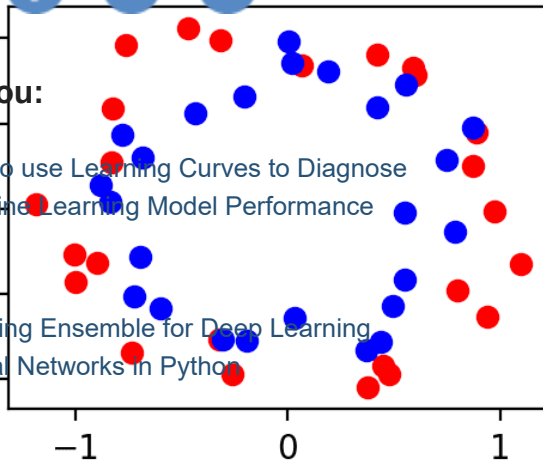
How To Improve Deep Learning Performance



How to use Data Scaling Improve Deep Learning Model Stability and Performance



Gentle Introduction to the Adam Optimization Algorithm for Deep Learning



Loving the Tutorials?

Four Scatter Plots of the Circles Dataset Varied by the Amount of Samples

The [Better Deep Learning](#) Ebook is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE

**Want Better Results with Deep Learning?**

Take my free 7-day email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Start Machine Learning

Download Your **FREE** mini-Course

## Neural Network Model Variance

We can model the circles problem with a neural network.

Specifically, we can train a Multilayer Perceptron model, or MLP, generated from the circles problem to the model. Once learned, the problem by using it to make predictions on new examples.

We can develop a small MLP for the problem using the [Keras](#) hidden layer, and one output. A rectified linear activation function

## Start Machine Learning



You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE



Because the problem is a binary classification problem, the model can use the sigmoid activation function on the output layer to predict the probability of a sample belonging to class 0 or 1.



The model can be trained using an efficient version of mini-batch stochastic gradient descent called **Adam**, where each weight in the model has its own adaptive learning rate. The binary cross entropy loss function can be used as the cost function for optimization, where smaller loss values indicate a better model fit.

```
1 # define model
2 model = Sequential()
3 model.add(Dense(25, input_dim=2, activation='relu'))
4 model.add(Dense(1, activation='sigmoid'))
5 model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```



Stacking Ensemble for Deep Learning

Require samples generated from the circles problem to train the model and a separate test set not used to train the model that can be used estimate how well the model might perform on average when making predictions on new data.



How To Improve Deep Learning

The `create_dataset()` function below will create these train and test sets of a given size and uses a default noise of 0.1.

```
1 # create a test dataset
2 def create_dataset(n_train, n_test, noise=0.1):
3     # generate samples
4     n_samples = n_train + n_test
5     X, y = make_circles(n_samples=n_samples, noise=noise, random_state=1)
6     # split into train and test
7     trainX, testX = X[:n_train, :], X[n_train:, :]
8     trainy, testy = y[:n_train], y[n_train:]
9     # return samples
10    return trainX, trainy, testX, testy
```

We can call this function to prepare the train and test sets split into input and output components.

### Loving the Tutorials?

```
1 # create dataset
2 trainX, trainy, testX, testy = create_dataset(500, 500)
```

where you'll find the **Really Good** stuff.

Once we have defined the datasets and the model, we can train the model. We will train the model for 500 epochs on the training set.

```
1 # fit model
2 history = model.fit(trainX, trainy, epochs=500, verbose=0)
```

Once fit, we use the model to make predictions for the input examples of the test set and compare them to the true output values of the test set, then calculate an accuracy score.

Start Machine Learning

The `evaluate()` function performs this operation returning both the loss and accuracy of the model on the test dataset. We can ignore the loss and display the accuracy to get an idea of how well the model has learned to map random examples from the circles domain to a class 0 or class 1.

```
1 # evaluate the model
2 _, test_acc = model.evaluate(testX, testy, verbose=0)
3 print('Test Accuracy: %.3f' % (test_acc*100))
```

Tying all of this together, the complete example is listed below.

```
1 # mlp evaluated on the circles dataset
2 from sklearn.datasets import make_circles
3 from keras.layers import Dense
4 from keras.models import Sequential
5
6 # create a test dataset
7 def create_dataset(n_train, n_test, noise=0.1):
8     # generate samples
9     n_samples = n_train + n_test
```

## Start Machine Learning



You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

```

10 X, y = make_circles(n_samples=n_samples, noise=noise, random_state=1)
11 # Split into train and test
12 trainX, testX = X[:n_train, :], X[n_train:, :]
13 trainy, testy = y[:n_train], y[n_train:]
14 # return samples
15 return trainX, trainy, testX, testy
16
17 # create dataset
18 trainX, trainy, testX, testy = create_dataset(500, 500)
19 # define model
20 model = Sequential()
21 model.add(Dense(25, activation='relu'))
22 model.add(Dense(1, activation='sigmoid'))
23 model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
24 # fit model
25 history = model.fit(trainX, trainy, epochs=500, verbose=0)
26 # evaluate the model
27 test_acc = model.evaluate(testX, testy, verbose=0)
28 print('Test Accuracy: %.3f' % (test_acc*100))

```

Running the example generates the dataset, fits the model on the training dataset, and evaluates the model on the test set.

**Note:** Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in your hardware. Consider running the example a few times and compare the average outcome.

In this case, the model achieved an estimated accuracy of about 84.4%.

1 Test Accuracy: 84.400

Stochastic Learning Algorithm

A problem with this example is that your results may vary.

### Loving the Tutorials?

You may see different estimated accuracy each time the example is run.

The [Better Deep Learning](#) EBook is

where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE

1 Test Accuracy: 84.800

The examples from the dataset are the same each time the example is run. This is because we fixed the pseudorandom number generator when creating the samples. The samples do have noise, but we are getting the same noise each time.

Neural networks are a nonlinear learning algorithm, meaning that they can learn complex nonlinear relationships between the input variables and the output variables. They can approximate challenging nonlinear functions.

As such, we refer to neural network models as having a low bias and a high variance. They have a low bias because the approach makes few assumptions about the mathematical relationship between the input and output. They have a high variance because they are sensitive to the specific examples in the training set. This means that different training examples can mean a very different resulting model.

Although neural networks are a high-variance low-bias method, they can achieve good performance when the same algorithm is run on the same generated data.

Instead, we are seeing a difference in performance across multiple runs of the example, which is due to the stochastic nature of the algorithm.

Start Machine Learning

## Start Machine Learning

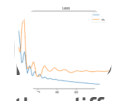
You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

**Never miss a tutorial:** The learning algorithm uses elements of randomness that aid the model in doing a good job on average of learning how to map input variables to output variables in the training dataset. Examples of randomness include the small random values used to initialize the model weights and the random sorting of examples in the training dataset before each training epoch.

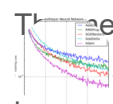
**Picked for you:** This useful randomness as it allows the model to automatically “*discover*” good solutions to the mapping function.

 How to use Learning Curves to Diagnose Machine Learning Model Performance  
The frustrating because it often finds different solutions each time the learning algorithm is run, and sometimes the differences between the solutions is large, resulting in differences in the estimated performance of the model when making predictions on new data.

 Stacking Ensemble for Deep Learning  
Neural Networks in Python  
**Average Model Performance**

We can counter the variance in the solution found by a specific neural network by summarizing the performance of the model over multiple runs.

This involves fitting the same algorithm on the same dataset multiple times but allowing the randomness used in the learning algorithm to vary each time the algorithm is run. The model is evaluated on the same test set each run and a score is recorded. At the end of all repeats, the distribution of the scores is summarized using a mean and standard deviation.

 The mean of the performance of a model over multiple runs gives an idea of the average performance of the model on a specific dataset. The spread or standard deviation of the scores gives an idea of the variance introduced by the learning algorithm.

We can move the evaluation of a single MLP to a function that takes the dataset and returns the accuracy on the test set. The `evaluate_model()` function below implements this behavior.

The Better Deep Learning EBook is

```
1 # evaluate an mlp model
2 def evaluate_model(trainX, trainy, testX, testy):
3     # define model
4     model = Sequential()
5     model.add(Dense(25, input_dim=2, activation='relu'))
6     model.add(Dense(1, activation='sigmoid'))
7     model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
8     # fit model
9     model.fit(trainX, trainy, epochs=500, verbose=0)
10    # evaluate the model
11    _, test_acc = model.evaluate(testX, testy, verbose=0)
12    return test_acc
```

Start Machine Learning

We can then call this function multiple times such as 10, 30, or more.

Given the [law of large numbers](#), more runs will mean a more accurate estimate. To keep running time modest, we will repeat the run 10 times.

```
1 # evaluate model
2 n_repeats = 10
3 scores = list()
4 for i in range(n_repeats):
5     # evaluate model
6     score = evaluate_model(trainX, trainy, testX, testy)
7     # store score
8     scores.append(score)
9     # summarize score for this run
10    print('>%d: %.3f' % (i+1, score*100))
```

Then, at the end of these runs, the mean and standard deviation of the scores can be used to summarize the distribution using a box and whisker plot via the

## Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.** Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

```

1 # report distribution of scores
2 mean_score, std_score = mean(scores)*100, std(scores)*100
3 print('Score Over %d Runs: %.3f (%.3f)' % (n_repeats, mean_score, std_score))
4 # plot distribution
5 pyplot.boxplot(scores)
6 pyplot.show()

```

Picked for you: If all of these elements together, an example of the repeated evaluation of an MLP model on the circles dataset is listed below.

How to use Learning Curves to Diagnose

```

1 # Machine Evaluating Model Performance on the circles dataset
2 from sklearn.datasets import make_circles
3 from keras.layers import Dense
4 from keras.models import Sequential
5 from sklearn.ensemble import GradientBoostingClassifier
6 from numpy import std
7 from matplotlib import pyplot
8
9 # create a test dataset
10 def create_dataset(n_train, n_test, noise=0.1):
11     # generate samples
12     n_samples = n_train + n_test
13     X, y = make_circles(n_samples=n_samples, noise=noise, random_state=1)
14     # split into train and test
15     trainX, testX = X[:n_train, :], X[n_train:, :]
16     trainy, testy = y[:n_train], y[n_train:]
17     return trainX, trainy, testX, testy
18
19 # evaluate an mlp model
20 def evaluate_model(trainX, trainy, testX, testy):
21     # define model
22     model = Sequential()
23     model.add(Dense(25, input_dim=2, activation='relu'))
24     model.add(Dense(1, activation='sigmoid'))
25     model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
26     # fit model
27     model.fit(trainX, trainy, epochs=500, verbose=0)
28     # evaluate the model
29     _, test_acc = model.evaluate(testX, testy, verbose=0)
30     return test_acc
31
32 # create dataset
33 trainX, trainy, testX, testy = create_dataset(500, 500)
34 # evaluate model
35 n_repeats = 10
36 scores = list()
37 for i in range(n_repeats):
38     # evaluate model
39     score = evaluate_model(trainX, trainy, testX, testy)
40     # store score
41     scores.append(score)
42     # summarize score for this run
43     print('>%d: %.3f' % (i+1, score*100))
44 # report distribution of scores
45 mean_score, std_score = mean(scores)*100, std(scores)*100
46 print('Score Over %d Runs: %.3f (%.3f)' % (n_repeats, mean_score, std_score))
47 # plot distribution
48 pyplot.boxplot(scores)
49 pyplot.show()

```

Running the example first reports the score of the model on each run.

**Note:** Your results may vary given the stochastic nature of the numerical precision. Consider running the example a few times.

At the end of the repeats, the average score is reported as above. That means for this specific model trained on the specific train 99% of runs will result in a test accuracy between 83.2% and 83.4% mean.

Start Machine Learning

## Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.** Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

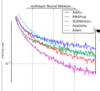
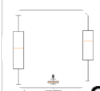
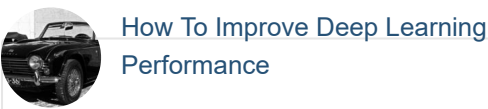
No doubt, the small sample size of 10 has resulted in some error in these estimates.

Never miss a tutorial:

|    |                                    |           |
|----|------------------------------------|-----------|
| 1  | >1:                                | 84.600    |
| 2  | >2:                                | 84.800    |
| 3  | >3:                                | 85.400    |
| 4  | >4:                                | 85.000    |
| 5  | >5:                                | 83.600... |
| 6  | >6:                                | 85.600    |
| 7  | >7:                                | 84.400    |
| 8  | >8:                                | 84.600    |
| 9  | >9:                                | 84.600    |
| 10 | >10:                               | 84.400    |
| 11 | Score Over 10 Runs: 84.700 (0.531) |           |

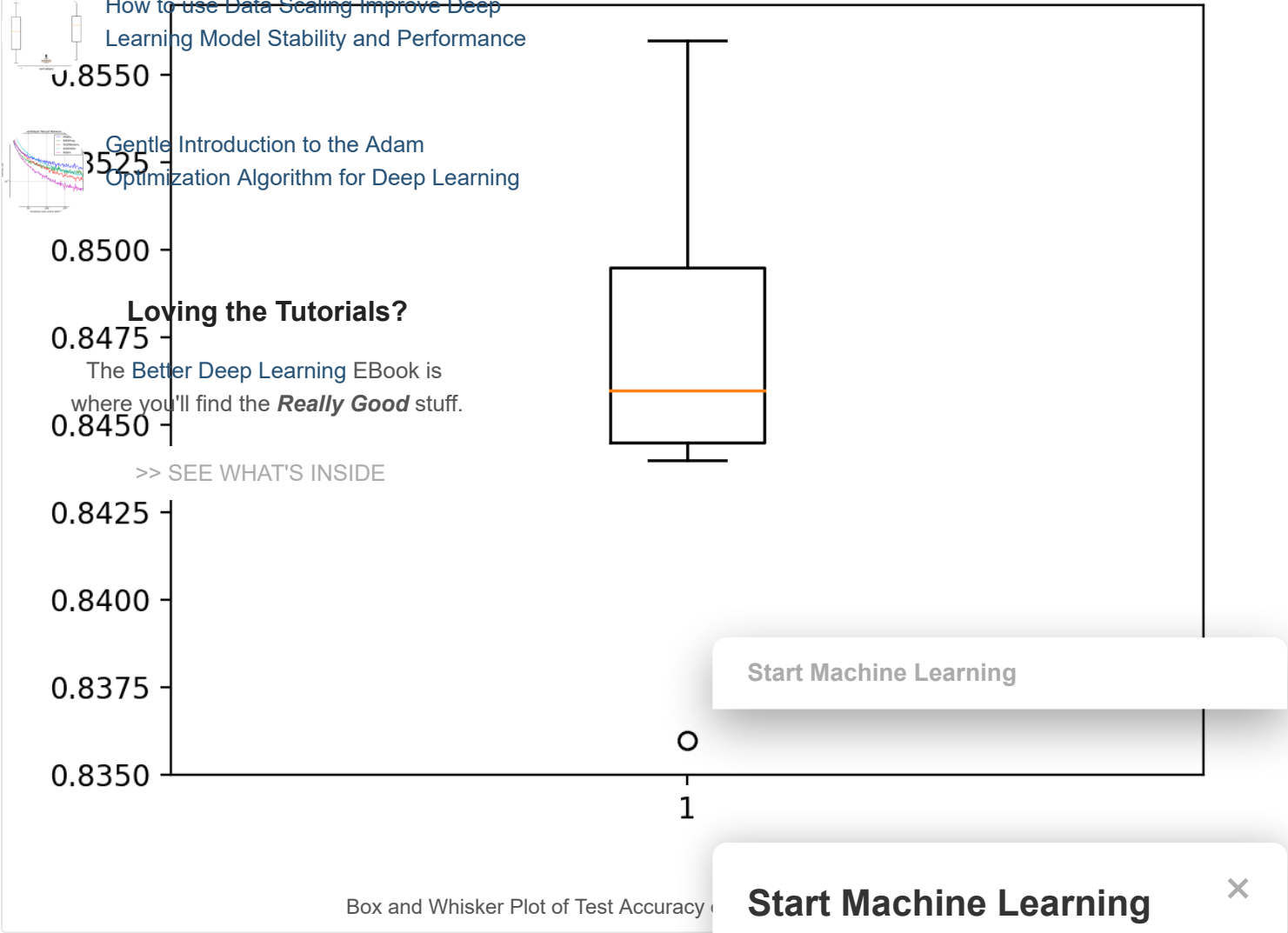
A box and whisker plot of the test accuracy scores is created showing the middle 50% of scores (called the interquartile range) denoted by the box ranges from a little below 84.5% to a little below 85%.

We can also see that the 83% value observed may be an outlier, given it is denoted as a dot.



**Loving the Tutorials?**  
The [Better Deep Learning](#) EBook is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



## Study Test Accuracy vs Training Set Size

Given a fixed amount of statistical noise and a fixed but reasonable amount of data, how much data is required to learn the circles problem?

We can investigate this question by evaluating the performance of the model on a test set of a different size.

### Start Machine Learning

You can master applied Machine Learning without math or fancy degrees. Find out how in this *free* and *practical* course.

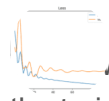
Email Address

START MY EMAIL COURSE



**Never miss a tutorial:** As a foundation, we can define a large number of examples that we believe would be far more than sufficient to learn the problem, such as 100,000. We can use this as an upper-bound on the number of training examples and use this as any number of examples in the test set. We will define a model that performs well on this dataset as a model that has effectively learned the two circles problem.

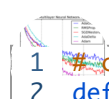
**Picked for you:** We can experiment by fitting models with different sized training datasets and evaluate their performance on the test set.

 [How to use Learning Curves to Diagnose Machine Learning Model Performance](#)  
A small number of training examples will result in a low test accuracy, perhaps because the chosen model overfits the training set or the training set is not sufficiently representative of the problem.

 [Stacking Ensemble for Deep Learning Neural Networks in Python](#)  
A large number of training examples will result in good, but perhaps slightly lower than ideal test accuracy, perhaps because the model does not have the capacity to learn the nuance of such a large training dataset, or the dataset is over-representative of the problem.

 [How To Improve Deep Learning Performance](#)  
A plot of training dataset size to model test accuracy should show an increasing trend to a point of diminishing returns and perhaps even a final slight drop in performance.

We can use the `create_dataset()` function defined in the previous section to create train and test datasets and set a size for the size of the test set argument to be 100,000 examples while allowing the size of the training set to be specified and vary with each call. Importantly, we want to use the same test set for each different sized training dataset.

 [Gentle Introduction to the Adam Optimization Algorithm for Deep Learning](#)

```
1 # create train and test datasets
2 def create_dataset(n_train, n_test=100000, noise=0.1):
3     # generate samples
4     n_samples = n_train + n_test
5     X, y = make_circles(n_samples=n_samples, noise=noise, random_state=1)
6     # split into train and test, first n for test
7     trainX, testX = X[n_test:, :], X[:n_test, :]
8     trainy, testy = y[n_test:], y[:n_test]
9     # return samples
10    return trainX, trainy, testX, testy
```

We can do `>> SEE WHAT'S INSIDE` `evaluate_model()` function from the previous section to fit and evaluate an MLP model on a given train and test set.

```
1 # evaluate an mlp model
2 def evaluate_model(trainX, trainy, testX, testy):
3     # define model
4     model = Sequential()
5     model.add(Dense(25, input_dim=2, activation='relu'))
6     model.add(Dense(1, activation='sigmoid'))
7     model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
8     # fit model
9     model.fit(trainX, trainy, epochs=500, verbose=0)
10    # evaluate the model
11    _, test_acc = model.evaluate(testX, testy, verbose=0)
12    return test_acc
```

We can create a new function to perform the repeated evaluation of the machine learning algorithm.

The `evaluate_size()` function below takes the size of the training dataset, the number of repeats, that defaults to five to keep running time down. The `create_dataset` and `evaluate_model` functions are called, and then the `evaluate_model()` function is called. The function returns a list of scores for the repeats.

```
1 # repeated evaluation of mlp model with dataset of a
2 def evaluate_size(n_train, n_repeats=5):
3     # create dataset
```

Start Machine Learning

## Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.** Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

```

4 trainX, trainy, testX, testy = create_dataset(n_train)
5 # repeated evaluation of model with dataset
6 scores = list()
7 for _ in range(n_repeats):
8     # evaluate model for size
9     score = evaluate_model(trainX, trainy, testX, testy)
10    scores.append(score)
11    return scores

```

This function can then be called repeatedly. I would guess that somewhere between 1,000 and 10,000 examples of the problem would be sufficient to learn the problem, where sufficient means only small fractional differences in test accuracy.

Therefore, we will investigate training set sizes of 100, 1,000, 5,000, and 10,000 examples. The mean test score will be reported for each test size to give an idea of progress.

```

1 # define dataset sizes to evaluate
2 sizes = [100, 1000, 5000, 10000]
3 score_sets, means = list(), list()
4 for n_train in sizes:
5     # repeated evaluate model with training set size
6     scores = evaluate_size(n_train)
7     score_sets.append(scores)
8     # summarize score for size
9     mean_score = mean(scores)
10    means.append(mean_score)
11    print('Train Size=%d, Test Accuracy %.3f' % (n_train, mean_score*100))

```

At the end of the run, a line plot will be created to show the relationship between train set size and model test set accuracy. We would expect to see an exponential curve from poor accuracy to a point of diminishing returns.

Box and whisker plots of the score distributions for each test set size are also created. We would expect to see a shrinking in the spread of test accuracy as the size of the training set is increased.

## Loving the Tutorials?

The complete example is listed below.  
The Better Deep Learning eBook is

```

1 # Study of training set size for an mlp on the circles problem
2 from sklearn.datasets import make_circles
3 from keras.layers import Dense
4 from keras.models import Sequential
5 from numpy import mean
6 from matplotlib import pyplot
7
8 # create train and test datasets
9 def create_dataset(n_train, n_test=100000, noise=0.1):
10     # generate samples
11     n_samples = n_train + n_test
12     X, y = make_circles(n_samples=n_samples, noise=noise)
13     # split into train and test, first n for test
14     trainX, testX = X[n_test:, :], X[:n_test, :]
15     trainy, testy = y[n_test:], y[:n_test]
16     # return samples
17     return trainX, trainy, testX, testy
18
19 # evaluate an mlp model
20 def evaluate_model(trainX, trainy, testX, testy):
21     # define model
22     model = Sequential()
23     model.add(Dense(25, input_dim=2, activation='relu'))
24     model.add(Dense(1, activation='sigmoid'))
25     model.compile(loss='binary_crossentropy', optimizer='adam')
26     # fit model
27     model.fit(trainX, trainy, epochs=500, verbose=0)
28     # evaluate the model
29     _, test_acc = model.evaluate(testX, testy, verbose=0)
30     return test_acc
31
32 # repeated evaluation of mlp model with dataset of a
33 def evaluate_size(n_train, n_repeats=5):

```

Start Machine Learning

## Start Machine Learning

You can master applied Machine Learning  
**without math or fancy degrees.**  
Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

```

34 # create dataset
35 trainX, trainY, testX, testY = create_dataset(n_train)
36 # repeat evaluation of model with dataset
37 scores = list()
38 for _ in range(n_repeats):
39     # evaluate model for size
40     score = evaluate_model(trainX, trainY, testX, testY)
41     scores.append(score)
42 return scores
43
44 # define dataset sizes to evaluate
45 sizes = [100, 1000, 5000, 10000]
46 score_sets, means = list(), list()
47 for n_train in sizes:
48     # repeated evaluate model with training set size
49     scores = evaluate_size(n_train)
50     score_sets.append(scores)
51     # summarize score for size
52     mean_score = mean(scores)
53     means.append(mean_score)
54     print('Train Size=%d, Test Accuracy %.3f' % (n_train, mean_score*100))
55 # summarize relationship of train size to test accuracy
56 pyplot.plot(sizes, means, marker='o')
57 pyplot.show()
58 # plot distributions of test accuracy for train size
59 pyplot.boxplot(score_sets, labels=sizes)
60 pyplot.show()

```

Learning Model Stability and Performance

Running the example may take a few minutes on modern hardware.

Now, your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in precision. Consider running the example a few times and compare the average outcome.

The mean model performance is reported for each training set size, showing a steady improvement in test accuracy as the training set is increased, as we expect. We also see a small drop in the average model performance from 5,000 to 10,000 examples, very likely highlighting that the variance in the data sample has exceeded the capacity of the chosen model configuration (number of layers and nodes).

The Better Deep Learning Ebook is

where you'll find the **Really Good** stuff.

```

1 Train Size=100, Test Accuracy 72.041
2 Train Size=1000, Test Accuracy 83.719
3 Train Size=5000, Test Accuracy 84.062
4 Train Size=10000, Test Accuracy 84.025

```

A line plot of test accuracy vs training set size is created.

We can see a sharp increase in test accuracy from 100 to 1,000 examples, after which performance appears to level off.

Start Machine Learning

## Start Machine Learning ×

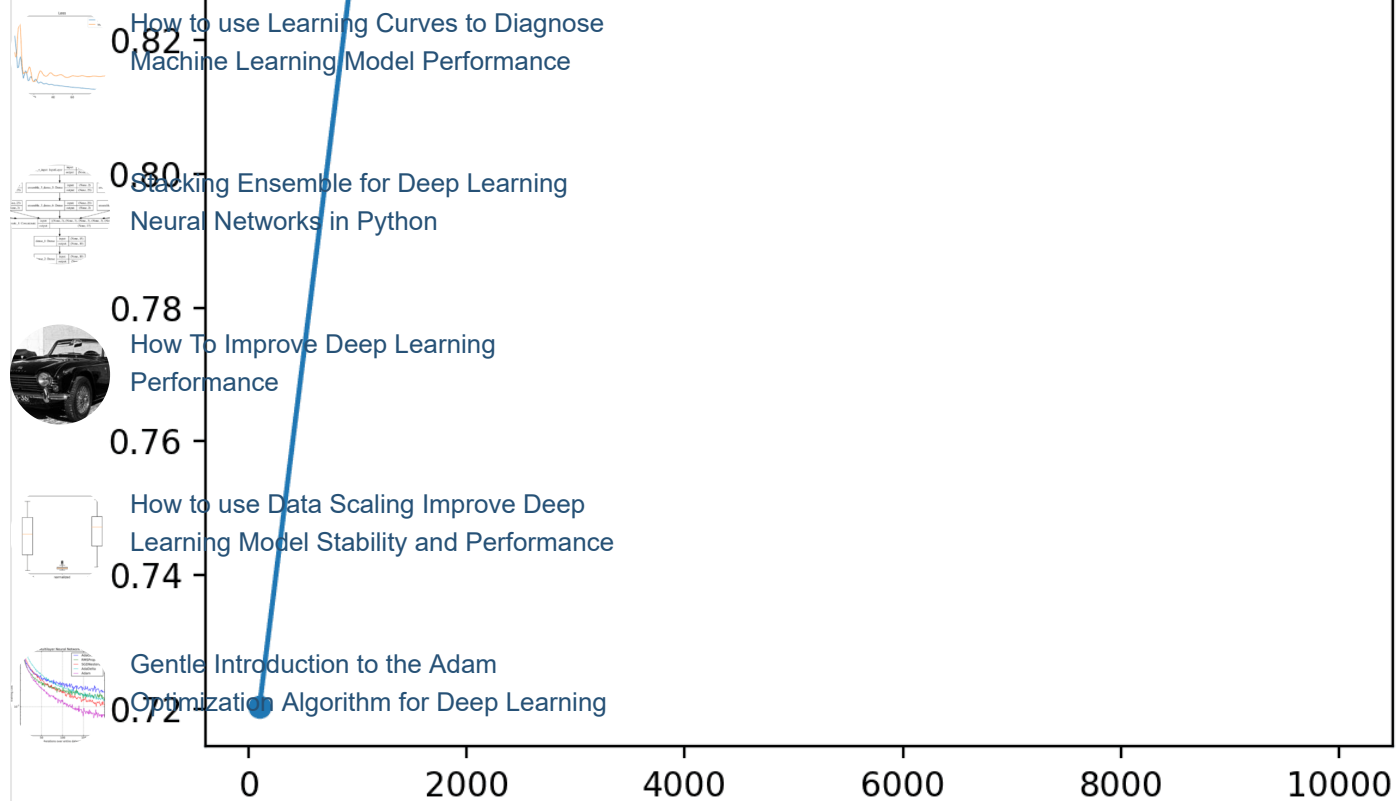
You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

Never miss a tutorial:



Picked for you:



### Loving the Tutorials?

The [Better Deep Learning](#) EBook is where you'll find the **Really Good** stuff.

A box and whisker plot is created showing the distribution of test accuracy scores for each sized training dataset. As expected, the spread of test set accuracy scores shrinks dramatically as the training set size is increased, although remains small on the plot given the chosen scale.

Start Machine Learning

## Start Machine Learning



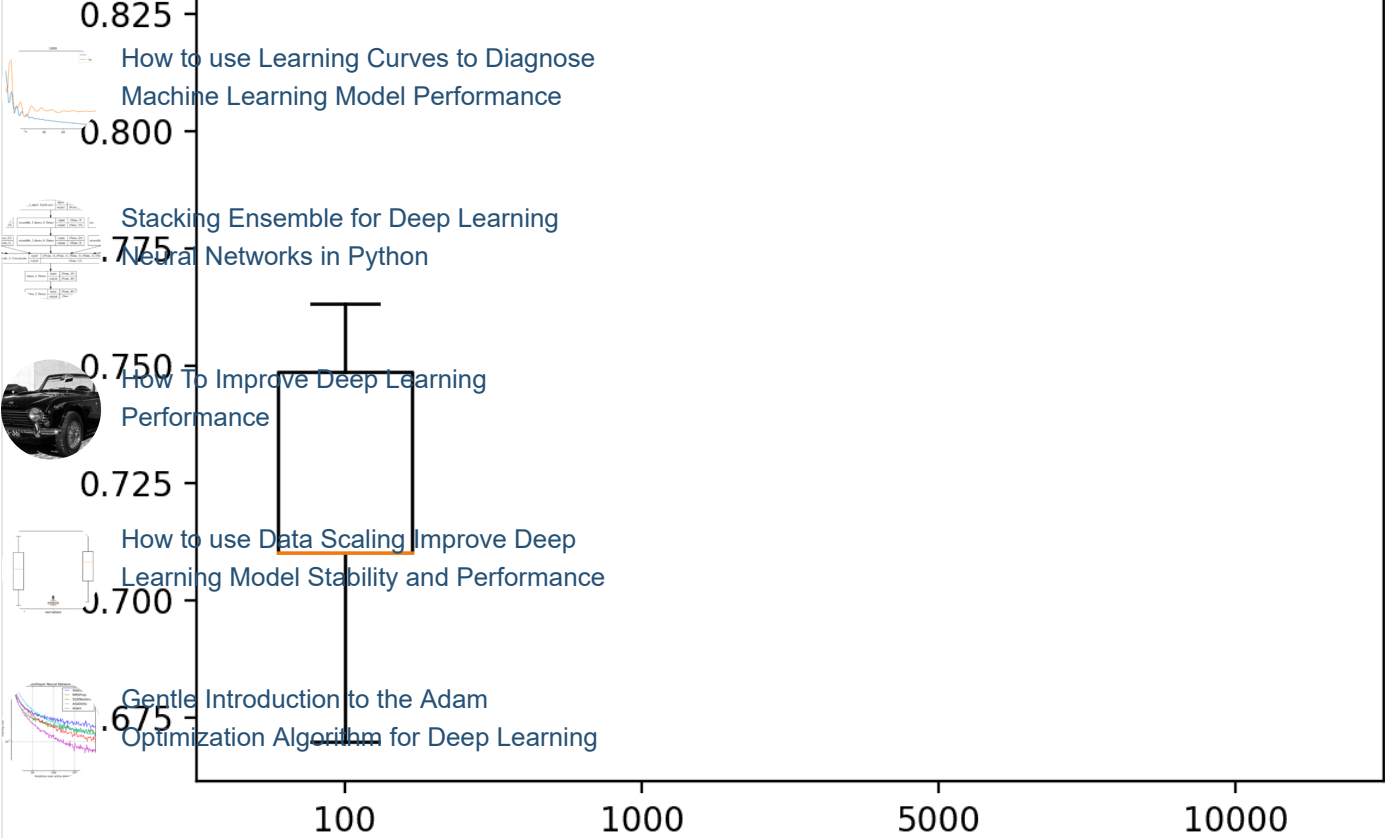
You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

Never miss a tutorial:



Picked for you:



Loving the Tutorials?

Box and Whisker Plots of Test Set Accuracy of MLPs Trained With Different Sized Training Sets on the Circles Problem  
The [Better Deep Learning](#) Ebook is

where you'll find the **Really Good** stuff  
The results suggest that the chosen MLP model configuration can learn the problem reasonably well with 1,000 examples >> SEE WHAT'S INSIDE  
ements seen with 5,000 and 10,000 examples. Perhaps there is a sweet spot of 2,500 Examples that results in an 84% test set accuracy with less than 5,000 examples.

The performance of neural networks can continually improve as more and more data is provided to the model, but the capacity of the model must be adjusted to support the increases in data. Eventually, there will be a point of diminishing returns where more data will not provide more insight into the underlying mapping function. For simpler problems, like two circles, this point will be reached sooner than for more complex problems like classifying photos of objects.

This study has highlighted a fundamental aspect of applied machine learning, specifically that you need enough examples of the problem to learn a useful approximation of the unknown underlying mapping function.

We almost never have an abundance of or too much training data, so it's important to know how to economically use available data and how to avoid overfitting to the training data.

Study Test Set Size vs Test Set Accuracy

Given a fixed model and a fixed training dataset, how much test data do you need to accurately estimate the model performance?

We can investigate this question by fitting an MLP with a fixed architecture and training it on different sized test sets.

Start Machine Learning

Start Machine Learning

You can master applied Machine Learning without math or fancy degrees. Find out how in this free and practical course.

Email Address

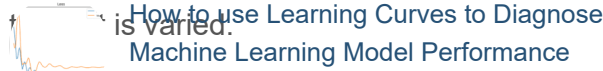
START MY EMAIL COURSE



We can use much the same strategy as the study in the previous section. We will fix the training set size at 1,000 examples as it resulted in a reasonably effective model with an estimated accuracy of about 83.7% when evaluated on 100,000 examples. We would expect that there is a smaller test set size that can reasonably approximate this value.

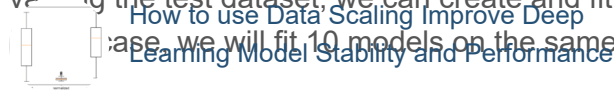


**Picked for you:** The `evaluate_test_set_size()` function can be updated to specify the test set size and to use a default of 1,000 examples for the training set size. Importantly, the same 1,000 examples are used for the training set each time the size of the test set is varied.



```
1 # create dataset
2 def create_dataset(n_test, n_train=1000, noise=0.1):
3     # generate samples
4     n_samples = n_train + n_test
5     X, y = make_circles(n_samples=n_samples, noise=noise, random_state=1)
6     # split into train and test, first n for test
7     trainX, testX = X[:n_train, :], X[n_train:, :]
8     trainy, testy = y[:n_train], y[n_train:]
9     # return samples
10    return trainX, trainy, testX, testy
```

We can use the same `fit_model()` function to fit the model. Because we're using the same training dataset and varying the test dataset, we can create and fit the models once and re-use them for each differently sized test set. In this case, we will fit 10 models on the same training dataset to simulate 10 repeats.



```
1 # create fixed training dataset
2 trainX, trainy, _, _ = create_dataset(10)
3 # Fit model for each repeat
4 n_repeats = 10
5 models = [fit_model(trainX, trainy) for _ in range(n_repeats)]
6 print('Fit %d models' % n_repeats)
```

Once fit, we can evaluate each of the models using a given sized test dataset. The `evaluate_test_set_size()` function below implements this behavior, returning a list of test set accuracy scores for the list of fit models and a given test set size.

The Better Deep Learning EBook is

```
1 # evaluate a test set of a given size on the fit models
2 def evaluate_test_set_size(models, n_test):
3     # create dataset INSIDE
4     _, _, testX, testy = create_dataset(n_test)
5     scores = list()
6     for model in models:
7         # evaluate the model
8         _, test_acc = model.evaluate(testX, testy, verbose=0)
9         scores.append(test_acc)
10    return scores
```

Start Machine Learning

We will evaluate four different sized test sets with 100, 1,000, 5,000, and 10,000 examples. We can then report the mean scores for each sized test set and create the same line and box and whisker plots.

```
1 # define test set sizes to evaluate
2 sizes = [100, 1000, 5000, 10000]
3 score_sets, means = list(), list()
4 for n_test in sizes:
5     # evaluate a test set of a given size on the model
6     scores = evaluate_test_set_size(models, n_test)
7     score_sets.append(scores)
8     # summarize score for size
9     mean_score = mean(scores)
10    means.append(mean_score)
11    print('Test Size=%d, Test Accuracy %.3f' % (n_test, mean_score))
12 # summarize relationship of test size to test accuracy
13 pyplot.plot(sizes, means, marker='o')
14 pyplot.show()
15 # plot distributions of test size to test accuracy
16 pyplot.boxplot(score_sets, labels=sizes)
17 pyplot.show()
```

## Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.** Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

Tying these elements together, the complete example is listed below.

Never miss a tutorial.

```
1 # study of test set size for an mlp on the circles problem
2 from sklearn.datasets import make_circles
3 from keras.layers import Dense
4 from keras.models import Sequential
5 from numpy import mean
6 from matplotlib import pyplot
7
8 # create dataset
9 def create_dataset(n_test, n_train=1000, noise=0.1):
10     # generate samples
11     n_samples = n_train + n_test
12     X, y = make_circles(n_samples=n_samples, noise=noise, random_state=1)
13     # split into train and test, first n for test
14     trainX, testX = X[:n_train, :], X[n_train:, :]
15     trainy, testy = y[:n_train], y[n_train:]
16     # return samples
17     return trainX, trainy, testX, testy
18
19 # fit an mlp model
20 def fit_model(trainX, trainy):
21     # define model
22     model = Sequential()
23     model.add(Dense(25, input_dim=2, activation='relu'))
24     model.add(Dense(1, activation='sigmoid'))
25     model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
26     # fit model
27     model.fit(trainX, trainy, epochs=500, verbose=0)
28     return model
29
30 # evaluate a test set of a given size on the fit models
31 def evaluate_test_set_size(models, n_test):
32     # create dataset
33     _, _, testX, testy = create_dataset(n_test)
34     scores = list()
35     for model in models:
36         # evaluate the model
37         _, test_acc = model.evaluate(testX, testy, verbose=0)
38         scores.append(test_acc)
39     return scores
40
41 # create fixed training dataset
42 trainX, trainy, _, _ = create_dataset(10)
43 # fit one model for each repeat
44 n_repeats = 10
45 models = [fit_model(trainX, trainy) for _ in range(n_repeats)]
46 print('Fit %d models' % n_repeats)
47 # define test set sizes to evaluate
48 sizes = [100, 1000, 5000, 10000]
49 score_sets, means = list(), list()
50 for n_test in sizes:
51     # evaluate a test set of a given size on the models
52     scores = evaluate_test_set_size(models, n_test)
53     score_sets.append(scores)
54     # summarize score for size
55     mean_score = mean(scores)
56     means.append(mean_score)
57     print('Test Size=%d, Test Accuracy %.3f' % (n_test, mean_score))
58 # summarize relationship of test size to test accuracy
59 pyplot.plot(sizes, means, marker='o')
60 pyplot.show()
61 # plot distributions of test size to test accuracy
62 pyplot.boxplot(score_sets, labels=sizes)
63 pyplot.show()
```

Running the example reports the test set accuracy for each different test set size on the models trained on the same dataset.

**Note:** Your results may vary given the stochastic nature of the numerical precision. Consider running the example a few times

Start Machine Learning

Start Machine Learning

You can master applied Machine Learning  
**without math or fancy degrees.**  
Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

in

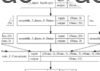
Never miss a tutorial: If we take the result from the previous section of 83.7% when evaluated on 100,000 examples as an estimate of ground truth, then we can see that the smaller sized test sets vary above and below this estimate.



- 1 Fit 10 models
- 2 Test Size=100, Test Accuracy 86.100
- 3 Test Size=1000, Test Accuracy 82.410
- 4 Test Size=5000, Test Accuracy 84.228
- 5 Test Size=10000, Test Accuracy 83.760

 How to use Learning Curves to Diagnose Machine Learning Model Performance

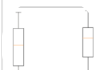
The plot shows that smaller test set sizes of 100 and 1,000 examples overestimate and underestimate the accuracy of the model respectively. The results for a test set of 5,000 and 10,000 are a closer fit, the latter perhaps the best match.

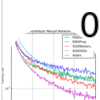
 Stacking Ensemble for Deep Learning

 Neural Networks in Python

This is surprising as a naive expectation might be that a test set of the same size as the training set would be a reasonable approximation of model accuracy, e.g. as though we performed a 50%/50% split of a 2,000 example

 How To Improve Deep Learning Performance

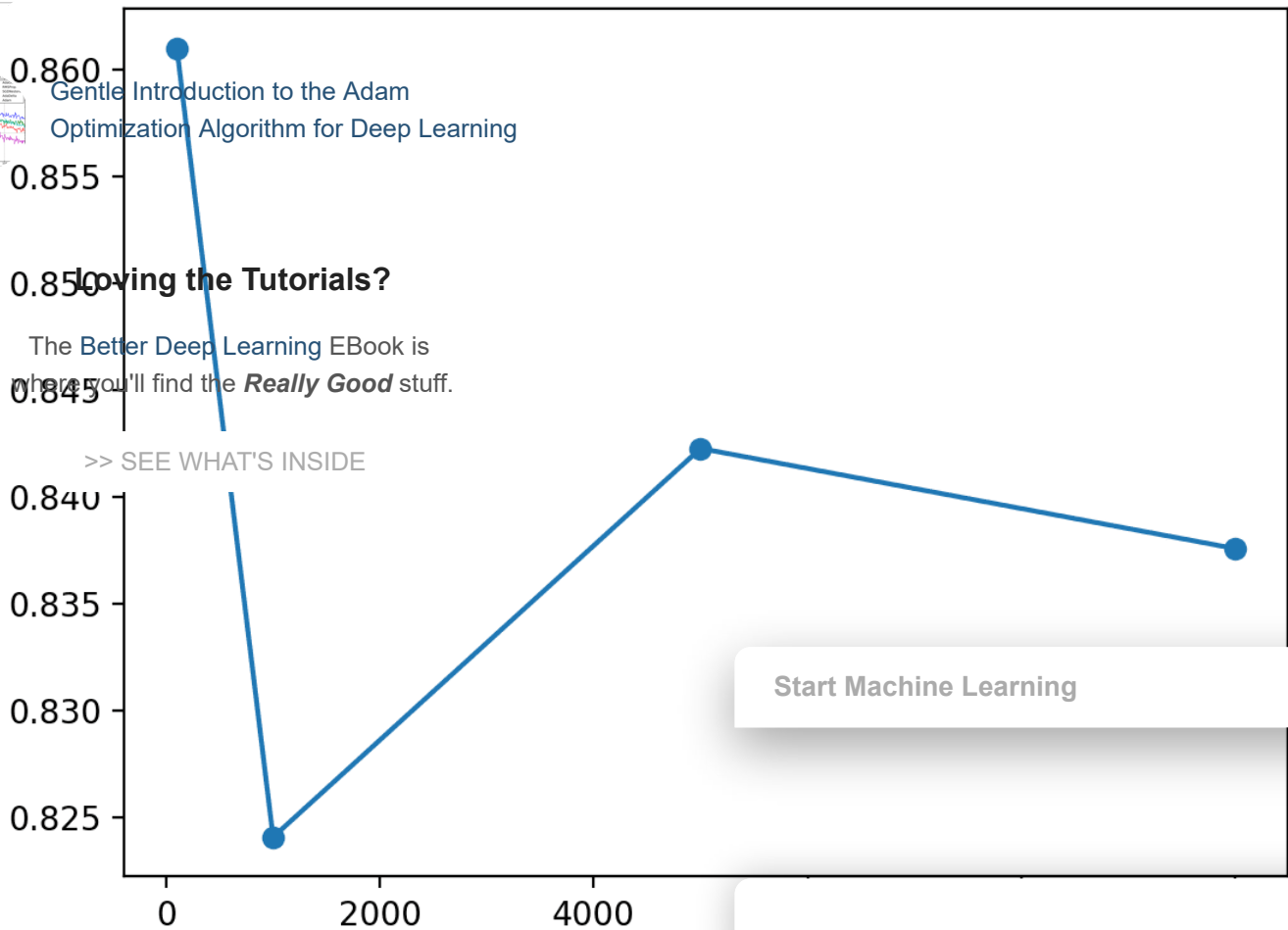
 How to use Data Scaling Improve Deep Learning Model Stability and Performance

 Gentle Introduction to the Adam Optimization Algorithm for Deep Learning

How to Improve the Tutorials?

The Better Deep Learning EBook is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



Line Plot of Test Set Size vs Test Set Accuracy

A box and whisker plots show that the smaller test set demonstrates more optimistic.

Start Machine Learning

## Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.** Find out how in this *free* and *practical* course.

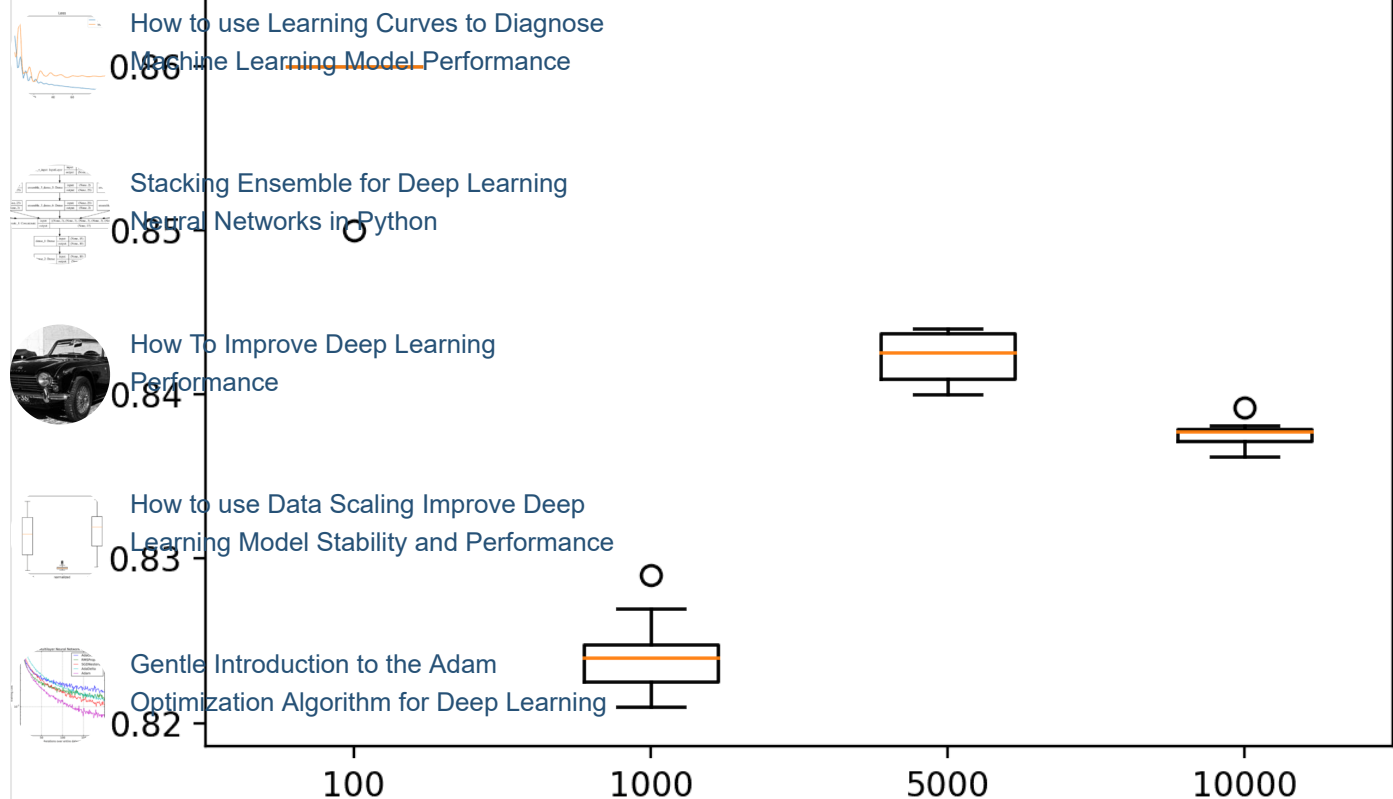
Email Address

START MY EMAIL COURSE

Never miss a tutorial:



Picked for you:



Loving the Tutorials?

The [Better Deep Learning](#) EBook is where you'll find the **Really Good** stuff. It should be pointed out that we are not reporting on the average performance of the chosen model on random examples >> SEE WHAT'S INSIDE The model and the examples are fixed and the only source of variation is from the learning algorithm.

The study demonstrates how sensitive the estimated accuracy of the model is to the size of the test dataset. This is an important consideration as often little thought is given to the test set size, using a familiar split of 70%/30% or 80%/20% for train/test splits.

As pointed out in the previous section, we often rarely have enough data, and spending a precious portion of the data on a test or even validation dataset is often rarely considered in any great depth. The results highlight how important cross-validation methods such as leave-one-out cross-validation are, allowing the model to make a test prediction for each example in the dataset, yet come at such a great computational cost, requiring almost one model to be trained for each example in the dataset.

In practice, we must struggle with a training dataset that is too function and a test dataset that is too small to reasonably apply to the problem.

## Further Reading

This section provides more resources on the topic if you are looking for more information.

- [sklearn.datasets.make\\_circles](#) API

### Start Machine Learning

### Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

• [matplotlib.pyplot.scatter API](#)

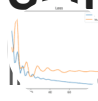
- [matplotlib.pyplot.subplot API](#)



- [Keras Sequential Model API](#)

**Picked for you:**

## Summary

 [How to use Learning Curves to Diagnose](#)

 [Machine Learning Model Performance](#)

In this tutorial, you discovered that in practice, we don't have enough data to learn the mapping function or to evaluate models, yet supervised learning algorithms like neural networks remain remarkably effective.

 [Stacking Ensemble for Deep Learning](#)

 [Neural Networks in Python](#)

- How to analyze the two circles classification problem and measure the variance introduced by the neural network learning algorithm.

 [How To Improve Deep Learning](#)

How changes in the size of a training dataset directly impact the quality of the mapping function approximated by neural networks.

- How the changes in the size of a test dataset directly impact the quality of the estimated in the performance of

 [How to use Data Scaling to Improve Deep](#)

[Learning Model Stability and Performance](#)

Do you have any questions?

Ask your questions in the comments below and I will do my best to answer.

 [Gentle Introduction to the Adam](#)

[Optimization Algorithm for Deep Learning](#)

## Develop Better Deep Learning Models Today!

### Loving the Tutorials?

The Better Deep Learning EBook is where you'll find the **Really Good** stuff.

Train Faster and More Accurately

>> SEE WHAT'S INSIDE

Jason Brownlee

MACHINE  
LEARNING  
MASTERY



### Train Faster, Reduce Overfitting, and Ensembles

...with just a few lines of python code

Discover how in my new Ebook:

[Better Deep Learning](#)

It provides **self-study tutorials** on topics like:

*weight decay, batch normalization, dropout, model stacking* and much more...

### Bring better deep learning to your projects!

Start [Machine Learning](#)

SEE WHAT'S INSIDE

### Start Machine Learning

You can master applied Machine Learning

**without math or fancy degrees.**

Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

Tweet

Share

Share



#### About Jason Brownlee

Jason Brownlee, PhD is a machine learning specialist who teaches machine learning methods via hands-on tutorials.

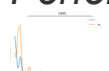
[View all posts by Jason Brownlee](#) →



Never miss a tutorial:



## Picked responses to *Impact of Dataset Size on Deep Learning Model Skill And Performance Estimates*

 [How to use Learning Curves to Diagnose Machine Learning Model Performance](#)

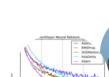
 **Dennis Cartin** January 9, 2019 at 1:48 am # REPLY ↩

 [Stacking Ensemble for Deep Learning Neural Networks in Python](#)  
Thanks Jason as always for your great article.

I have a question on small data set on binary classification, what is the best pre-processing implementation we would use?

 [How To Improve Deep Learning Performance](#)  
Do we need to zero centering our data or just normalisation of our data set?  
I pardon my question if not clear enough.

 Thanks  
[How to use Data Scaling Improve Deep Learning Model Stability and Performance](#)

  **Jason Brownlee** January 9, 2019 at 8:47 am # REPLY ↩  
[A Gentle Introduction to the Adam Optimization Algorithm for Deep Learning](#)  
Try a range of preprocessing steps and see which results in faster learning or better performance.

 **Loving the Tutorials?**  
**Jonathan Lastrilla** January 16, 2019 at 10:52 pm # REPLY ↩  
The [Better Deep Learning](#) EBook is  
where you will find the [Really Cool stuff](#).  
I have used the [Really Cool stuff](#) records of transactions for 2 years, I decided to use the last two months from that distribution as test set. and the remaining for train and dev sets. Is that a good idea? I am also aware of the possible >> SEE WHAT'S INSIDE test set. The problem is for multi-label classification.  
Thanks!

 **Jason Brownlee** January 17, 2019 at 5:25 am #  
I recommend testing different amount of history input to discover what works best for your specific dataset.

 **Grzegorz Kępiś** January 22, 2020 at 6:13 pm #  
Good morning Jason,  
Thank you for this post!  
Is there any specific reason why there is no possibility to set a p  
ANN? (Just as we do for data splitting, or for many ML algorithm  
seed and SGD as well. Do you know the reason why the imple  
Best regards!

Start Machine Learning

### Start Machine Learning

You can master applied Machine Learning  
**without math or fancy degrees.**  
Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

Never miss a tutorial:

Jason Brownlee

January 23, 2020 at 6:28 am #

REPLY ↩

Yes, it is a bad strategy:  
<https://machinelearningmastery.com/randomness-in-machine-learning/>

Picked for you:

Instead, we control for the stochastic nature of the learning algorithm using repeated model evaluations.

<https://machinelearningmastery.com/evaluate-skill-deep-learning-models/>

Machine Learning Model Performance

Starting Tutorial for Deep learning

Neural Networks in Python

Hi

REPLY ↩

Thanks so much for this detailed tutorial.  
[How To Improve Deep Learning Model Stability and Performance](#)  
I please request any information that you may have on dimensions vs accuracy and at what point the gains in accuracy diminish with regards to dimensions?

Thanks

How to use Data Scaling Improve Deep Learning Model Stability and Performance

Gentle Introduction to the Adam Optimization Algorithm for Deep Learning

Jason Brownlee

February 3, 2020 at 5:43 am #

REPLY ↩

It depends on the model and dataset.

You can run a study with your own model and dataset to discover the relationship.

Loving the Tutorials?

The [Better Deep Learning](#) EBook is where you'll find the **Really Good** stuff

Beny Nugraha

January 19, 2021 at 8:32 pm #

REPLY ↩

>> SEE WHAT'S INSIDE  
tion between the number of features with the DL model's training or testing time?  
Will training a model with 50 features take longer than training a model with 10 features? Thanks

Jason Brownlee

January 20, 2021 at 5:41 am #

Yes, more features means a slower model.

Start Machine Learning

danny

February 13, 2021 at 12:28 am #

Hello,

I wanted to ask for binary classification for 2000 samples. my algorithm trains very fast on jupyter notebook and it trains within one minute for 50 epochs.. i am using keras. time to train...i am new to this topic so pardon me if i

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

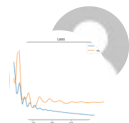
Email Address

START MY EMAIL COURSE

Never miss a tutorial: What is the problem exactly?

It trains fast?  
Achieves 90% accuracy?

## Picked for you:



**Akash** March 23, 2021 at 2:43 am #  
[How to use Learning Curves to Diagnose Machine Learning Model Performance](#)  
I'm currently working on an image classification problem (binary). I have an equal amount of images for both classes, but in reality, the possibility of the occurrence of one of the classes is much lower than the other. Is there a way to reflect this in the training process? Additionally, I also have the same question as most others here: How to find out how much data is enough to prevent overfitting? At present, I have ~250 original images for each class and I've already implemented augmentation to triple the size to ~750 in each class. How to find out how many more images are required? Thanks in advance.

REPLY ↩

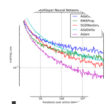


[How To Improve Deep Learning Performance](#)



**Jason Brownlee** March 23, 2021 at 4:58 am #  
[How to use Data Scaling Improve Deep Learning Model Stability and Performance](#)  
Yes, ideally the training dataset and test dataset would be representative of the underlying problem. Perhaps you can gain a different sample that better matches a random sample from the domain?

REPLY ↩



[Gentle Introduction to the Adam Optimization Algorithm for Deep Learning](#)

Leave a Reply

## Loving the Tutorials?

The [Better Deep Learning](#) EBook is where you'll find the ***Really Good*** stuff.

>> SEE WHAT'S INSIDE

Name (required)

Email (will not be published) (required)

Website

SUBMIT COMMENT



**Welcome!**  
I'm *Jason Brownlee* PhD and I **help developers** get results with **machine learning**.  
[Read more](#)

Start Machine Learning

## Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

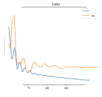
Email Address

START MY EMAIL COURSE

Never miss a tutorial:



## Picked for you:



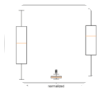
How to use Learning Curves to Diagnose Machine Learning Model Performance



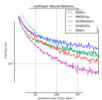
Stacking Ensemble for Deep Learning Neural Networks in Python



How To Improve Deep Learning Performance



How to use Data Scaling Improve Deep Learning Model Stability and Performance



Gentle Introduction to the Adam Optimization Algorithm for Deep Learning

## Loving the Tutorials?

The [Better Deep Learning](#) EBook is where you'll find the ***Really Good*** stuff.

>> SEE WHAT'S INSIDE

© 2021 Machine Learning Mastery Pty. Ltd. All Rights Reserved.  
[LinkedIn](#) | [Twitter](#) | [Facebook](#) | [Newsletter](#) | [RSS](#)

[Privacy](#) | [Disclaimer](#) | [Terms](#) | [Contact](#) | [Sitemap](#) | [Search](#)

Start Machine Learning

## Start Machine Learning



You can master applied Machine Learning **without math or fancy degrees.**  
Find out how in this *free* and *practical* course.

START MY EMAIL COURSE